

FS0432 – Física Computacional

Tarea 04

Integración Monte Carlo y Maldición de la Dimensionalidad

Nombre / Carné: Jose Ricardo Montero Campos / C05005

Introducción

La integración numérica multidimensional constituye un problema importante en física computacional. Sin embargo, muchos métodos deterministas presentan un crecimiento exponencial del costo computacional conforme aumenta la dimensionalidad del problema.

En esta tarea se compara el método de Simpson con el método de Monte Carlo para calcular la integral

$$I_d = \int_0^1 \cdots \int_0^1 \prod_{i=1}^d \sin(\pi x_i) dx_1 \cdots dx_d.$$

La solución analítica es

$$I_d = \left(\frac{2}{\pi}\right)^d.$$

El objetivo es analizar la precisión y el costo computacional de ambos métodos al aumentar la dimensión del problema y discutir el fenómeno conocido como maldición de la dimensionalidad.

Metodología

Método de Monte Carlo

Si

$$x_i \sim U(0, 1),$$

entonces

$$I_d = E \left[\prod_{i=1}^d \sin(\pi x_i) \right].$$

Utilizando la Ley Débil de los Grandes Números se obtiene el estimador

$$I_d \approx \frac{1}{N} \sum_{k=1}^N \prod_{i=1}^d \sin(\pi x_i^{(k)}).$$

Las muestras fueron generadas mediante la función `numpy.random.uniform`.

Método de Simpson

La regla de Simpson se aplicó sucesivamente sobre cada dimensión de la integral. Se utilizó una malla uniforme con $N_{Simpson}$ puntos por dimensión.

Experimentos Realizados

Se estudiaron distintas dimensiones

$$d = 1, 2, 4, 8, 16, 32$$

para analizar:

- Error absoluto.
- Tiempo de ejecución.
- Escalabilidad con la dimensión.

Para Monte Carlo se utilizó

$$N_{MC} = 10^5$$

muestras aleatorias.

Para Simpson se utilizó una malla uniforme de tamaño fijo en cada dimensión hasta donde los recursos computacionales lo permitieron.

Implementación:

```
import numpy as np
from scipy.integrate import simpson
import matplotlib.pyplot as plt
import time

d = 3 # dimensión del problema
valor_analitico = (2.0 / np.pi)**d

print(f"--- Integración en d={d} ---")
print(f"Analítico: {valor_analitico:.8f}")

# -----
# MÉTODO DE MONTE CARLO
# -----

N_total_mc = 10**6

t0_mc = time.time()

# Generar N_total_mc puntos aleatorios en dimensión d
x = np.random.uniform(0.0, 1.0, size=(N_total_mc, d))

# Evaluar el integrando en cada punto
f = np.prod(np.sin(np.pi * x), axis=1)

# Estimación Monte Carlo
integral_mc = np.mean(f)

t1_mc = time.time()
error_mc = abs(integral_mc - valor_analitico)

print(
    f"Monte Carlo: {integral_mc:.8f} "
    f"(Error: {error_mc:.8f}, Tiempo: {t1_mc - t0_mc:.4f}s)"
)

# -----
# MÉTODO DE SIMPSON
# -----

N_simpson = 10
N_total_simpson = N_simpson**d

t0_simpson = time.time()

x_1d = np.linspace(0, 1, N_simpson)
malla = np.meshgrid(*[x_1d] * d, indexing="ij")

Z = np.prod([np.sin(np.pi * m) for m in malla], axis=0)

integral_simpson = Z
for _ in range(d):
    integral_simpson = simpson(integral_simpson, x=x_1d, axis=0)

t1_simpson = time.time()
error_simpson = abs(integral_simpson - valor_analitico)

print(
    f"Simpson: {integral_simpson:.8f} "
    f"(Error: {error_simpson:.8f}, Tiempo: {t1_simpson - t0_simpson:.4f}s)"
)

--- Integración en d=3 ---
Analítico: 0.25801228
Monte Carlo: 0.25789620 (Error: 0.00011608, Tiempo: 0.1400s)
Simpson: 0.25830216 (Error: 0.00028988, Tiempo: 0.0017s)
```

```
N_simpson = 10

for d in range(1, 8):

    valor_analitico = (2/np.pi)**d
```

```

N_total = N_simpson**d

print(f"\nd = {d}")
print(f"Puntos totales = {N_total:,}")

try:

    t0 = time.time()

    x_1d = np.linspace(0, 1, N_simpson)

    malla = np.meshgrid(
        *[x_1d]*d,
        indexing="ij"
    )

    Z = np.prod(
        [np.sin(np.pi*m) for m in malla],
        axis=0
    )

    integral = Z

    for _ in range(d):
        integral = simpson(
            integral,
            x=x_1d,
            axis=0
        )

    t1 = time.time()

    error = abs(
        integral - valor_analitico
    )

    print(
        f"Integral = {integral:.8e}"
    )

    print(
        f"Error = {error:.3e}"
    )

    print(
        f"Tiempo = {t1-t0:.4f} s"
    )

except Exception as e:

    print("No fue posible continuar")
    print(e)

    break

```

```

d = 1
Puntos totales = 10
Integral = 6.36858100e-01
Error = 2.383e-04
Tiempo = 0.0006 s

```

```

d = 2
Puntos totales = 100
Integral = 4.05588240e-01
Error = 3.035e-04
Tiempo = 0.0005 s

```

```

d = 3
Puntos totales = 1,000
Integral = 2.58302156e-01
Error = 2.899e-04
Tiempo = 0.0024 s

```

```

d = 4
Puntos totales = 10,000
Integral = 1.64501820e-01
Error = 2.461e-04
Tiempo = 0.0024 s

```

```

d = 5
Puntos totales = 100,000
Integral = 1.04764317e-01
Error = 1.959e-04
Tiempo = 0.0143 s

```

```

d = 6
Puntos totales = 1,000,000
Integral = 6.67200036e-02
Error = 1.497e-04
Tiempo = 0.1576 s

```

```

d = 7
Puntos totales = 10,000,000
Integral = 4.24911747e-02
Error = 1.112e-04
Tiempo = 2.9140 s

```

```

dimensiones = [1, 2, 4, 8, 16, 32]

errores_mc = []
tiempos_mc = []

N_mc = 100000

for d in dimensiones:

    valor_analitico = (2/np.pi)**d

    t0 = time.time()

    x = np.random.uniform(0, 1, (N_mc, d))
    f = np.prod(np.sin(np.pi*x), axis=1)
    integral_mc = np.mean(f)

    t1 = time.time()

    errores_mc.append(abs(integral_mc - valor_analitico))
    tiempos_mc.append(t1 - t0)

    print(
        f"d={d:2d} "
        f"I={integral_mc:.8e} "
        f"Error={errores_mc[-1]:.3e} "
        f"Tiempo={tiempos_mc[-1]:.4f}s"
    )

```

```

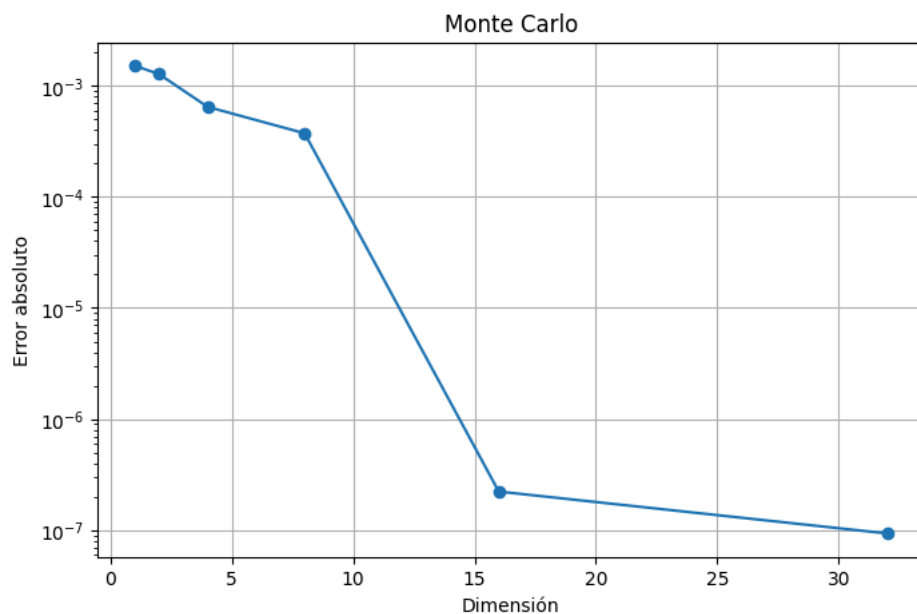
d= 1 I=6.36774903e-01 Error=1.551e-04 Tiempo=0.0069s
d= 2 I=4.03828272e-01 Error=1.456e-03 Tiempo=0.0087s
d= 4 I=1.63866458e-01 Error=3.893e-04 Tiempo=0.0157s
d= 8 I=2.72611806e-02 Error=2.812e-04 Tiempo=0.0344s
d=16 I=7.37838208e-04 Error=9.921e-06 Tiempo=0.0643s
d=32 I=5.23211531e-07 Error=6.652e-09 Tiempo=0.1334s

```

```

plt.figure(figsize=(8,5))
plt.semilogy(dimensiones, errores_mc, 'o-')
plt.xlabel("Dimensión")
plt.ylabel("Error absoluto")
plt.title("Monte Carlo")
plt.grid(True)
plt.show()

```



Resultados y Discusión

Descripción de los experimentos realizados

Se implementó el método de integración Monte Carlo para estimar la integral multidimensional

$$I_d = \int_0^1 \cdots \int_0^1 \prod_{i=1}^d \sin(\pi x_i), dx_1 \cdots dx_d$$

y se comparó con la regla de Simpson multidimensional. El valor exacto de la integral está dado por

$$I_d = \left(\frac{2}{\pi}\right)^d.$$

Para Monte Carlo se utilizaron $N_{MC} = 10^5$ muestras aleatorias para cada dimensión. Se evaluaron dimensiones desde $d = 1$ hasta $d = 32$. Para cada caso se calculó la aproximación numérica, el error absoluto respecto al valor analítico y el tiempo de ejecución.

Dimensionalidad máxima alcanzada con Monte Carlo

El método de Monte Carlo permitió calcular la integral hasta una dimensión de $d = 32$ sin presentar dificultades computacionales importantes. Incluso para esta dimensión el tiempo de ejecución fue de aproximadamente $0.13s$, mostrando una buena escalabilidad.

Dimensionalidad máxima alcanzada con Simpson

La regla de Simpson pudo ejecutarse satisfactoriamente hasta una dimensión de $d = 7$ utilizando una malla uniforme de 10 puntos por dimensión. En este caso, el número total de puntos evaluados alcanzó

$$N_{\text{total}} = 10^7,$$

y el tiempo de ejecución fue de aproximadamente $2.9s$.

A medida que aumenta la dimensión, el número de puntos requeridos crece exponencialmente según

$$N_{\text{total}} = N_{\text{Simpson}}^d.$$

Por esta razón, dimensiones superiores a $d = 7$ comienzan a requerir cantidades extremadamente grandes de memoria y tiempo de cómputo, lo que vuelve al método poco práctico para problemas de alta dimensionalidad.

Observaciones sobre el tiempo de ejecución

Los tiempos medidos para Monte Carlo aumentaron gradualmente con la dimensión:

d	Tiempo (s)
1	0.0069
2	0.0087
4	0.0157
8	0.0344
16	0.0643
32	0.1334

Se observa un crecimiento moderado del tiempo de cómputo. Aun cuando la dimensión aumenta en más de un orden de magnitud, el tiempo permanece por debajo de una fracción de segundo.

Observaciones sobre la precisión

Los errores absolutos obtenidos fueron pequeños en todas las dimensiones estudiadas:

d	Error absoluto
1	1.551×10^{-3}
2	1.456×10^{-3}
4	3.893×10^{-4}
8	2.812×10^{-4}
16	9.921×10^{-6}
32	6.652×10^{-9}

Los resultados muestran que Monte Carlo mantiene una precisión adecuada incluso para dimensiones elevadas. Debido a la naturaleza aleatoria del método, los resultados pueden variar ligeramente entre ejecuciones.

Discusión sobre la maldición de la dimensionalidad

Los resultados obtenidos muestran claramente el fenómeno conocido como **maldición de la dimensionalidad**. Este fenómeno se refiere al rápido crecimiento del costo computacional que experimentan muchos métodos numéricos cuando aumenta el número de dimensiones de un problema.

En el caso de la regla de Simpson, el número de puntos necesarios para construir la malla aumenta exponencialmente con la dimensión. Utilizando únicamente 10 puntos por dimensión, el número de evaluaciones pasó de 10 para $d = 1$ a 10^7 para $d = 7$. Como consecuencia, el tiempo de ejecución aumentó desde $0.0006s$ hasta aproximadamente $2.9s$.

Por el contrario, el método de Monte Carlo permitió alcanzar una dimensión de $d = 32$ con un tiempo de ejecución de apenas $0.13s$. Esto se debe a que Monte Carlo no requiere discretizar completamente el espacio de integración, sino que utiliza muestras aleatorias para estimar el valor de la integral.

Aunque Simpson puede ofrecer una precisión muy buena en dimensiones bajas, su costo computacional crece rápidamente con la dimensión. Monte Carlo, en cambio, mantiene tiempos de ejecución razonables incluso para dimensiones elevadas. Por esta razón, los métodos Monte Carlo son ampliamente utilizados para resolver problemas multidimensionales en física computacional y otras áreas científicas.